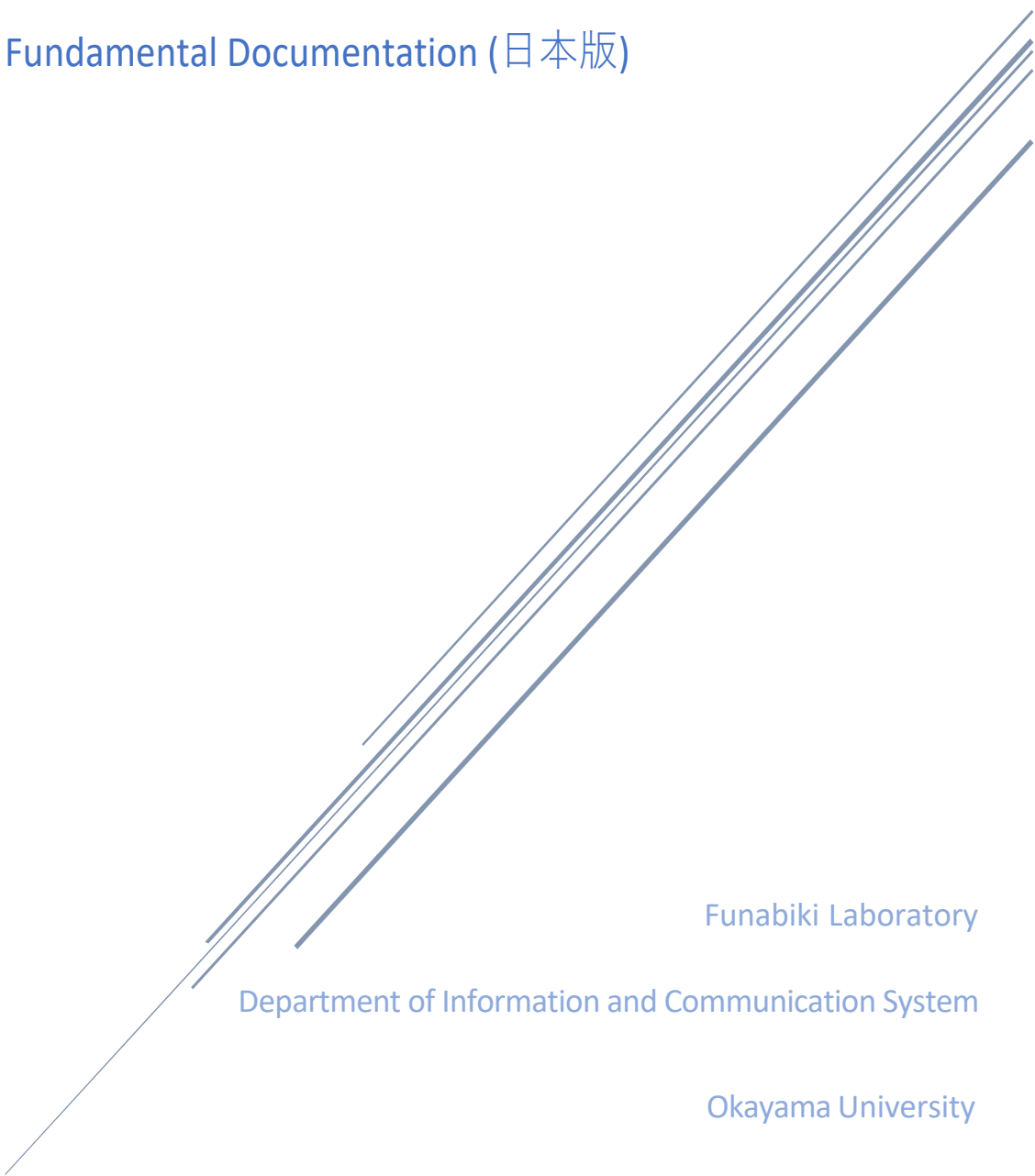


STUDY OF FLUTTER PROGRAMMING LEARNING ASSISTANT SYSTEM

Flutter Fundamental Documentation (日本版)



Funabiki Laboratory

Department of Information and Communication System

Okayama University

目次

各トピックと Flutter_INT 問題の対応表	3
Flutter と Dart プログラミング	4
Dart プログラミングの基本概念	4
Flutter 入門	8
なぜ Flutter はウィジェットを使うのか：簡単な説明	8
Flutter は古い UI フレームワークとどう違うのか？	9
まとめ	10
基本概念：ウィジェットはどのように機能するのか？	11
Flutter で Key-Value 形式を理解する	11
Flutter でウィジェットがネストされる理由	13
なぜ Flutter はウィジェット内部で関数やプロパティを使うのか？	14
まとめ	16
Flutter と Dart のプロジェクト構造の紹介	17
構造の分解（ステップごとの解説）	18
ステップ 1：インポート ‘package:flutter/material.dart’ の説明	18
ステップ 2：Flutter の main.dart の基本を理解する	19
ステップ 3：MyApp とウィジェットツリーを理解する	20
ステップ 4：クラスとエクステンンドを理解する	22
ステップ 5：@override を使ったメソッドのオーバーライド	23
ステップ 6：build() メソッドとBuildContext の探索	24
ステップ 7：Flutter における return の役割	25
ステップ 8：Flutter の MaterialApp とは？	26
ステップ 9：Flutter の Scaffold とは？	28
まとめ	30
Stateless vs Stateful ウィジェット	31
StatelessWidget と StatefulWidget を理解する	31
Flutter で setState を理解する	34

まとめ	36
ウィジェットの階層構造と発展的な概念	37
Flutter のウィジェット階層を理解する	37
第一級オブジェクトとしての関数と引数としての使用	39
三項条件操作	41
まとめ	42
よく使われるウィジェット	44
Flutter の AppBar ウィジェット	44
Flutter の Text ウィジェット	46
Flutter の Container ウィジェット	47
Flutter の Row と Column ウィジェット	50
Flutter のボタンウィジェット	52
まとめ	54

各トピックと Flutter_INT 問題の対応表

※リンクをクリックすると該当ページへ直接移動できます

Flutter の概念	関連する問題番号
Dart の演算子	10, 11, 12, 16
Dart のデータ型	10, 11, 12, 13, 14, 15, 16
ウィジェットの基本	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
Key-Value 形式の理解	3, 4, 5
ウィジェット内の関数とプロパティ	6, 8, 12
import パッケージ	1
main 関数	1, 6, 7, 12, 14, 15
MyApp とウィジェットツリー	2, 4
クラスと extends	2, 3, 6, 8, 9, 12, 15, 16
override キーワード	2, 10, 16
build メソッド	2, 3, 5, 10, 12, 14, 15, 16
BuildContext	2, 3, 5, 10, 12, 14, 15, 16
return 文	2, 4, 5, 12, 13, 16
MaterialApp	3, 4, 5, 7, 8, 9, 10, 12, 16
Scaffold ウィジェット	5, 6, 7, 8, 9, 11, 12, 14, 15, 16
Stateless と Stateful の違い	13, 15
setState メソッド	15
汎用関数（再利用関数）	14
三項条件演算子	16
AppBar ウィジェット	7, 11, 12, 13
Text ウィジェット	4, 6, 7, 13
Container ウィジェット	8
Row・Column ウィジェット	9, 11, 14, 15, 16
Button ウィジェット	10, 13

Flutter と Dart プログラミング

Flutter は、美しいユーザーインターフェース (UI) を持つアプリを、Android と iOS の両方で動作するように簡単に開発できるフレームワーク。

Flutter の裏側には Dart というプログラミング言語が使われている。

Dart はシンプルで高速、そして初心者にも学びやすいように設計されている。

Dart プログラミングの基本概念

1. Dart 変数

- 情報 (数字やテキスト) を保存するために使う。
- `var`, `int`, `double`, `String` などの型で宣言できる。

```
var name = 'Alice'; // A string variable
int age = 30;       // An integer variable
```

2. Dart の演算子

- 演算子は、変数や値に対して加算、減算、比較、代入などの操作を行うために使用される。
 - ・ より多くの演算子や使い方については、[こちら](#)をご覧ください。

```
int sum = 5 + 3; // Addition
bool isEqual = (5 == 5); // Comparison
```

3. Dart のキーワード

- キーワードとは、`if`, `for`, `class`, `void` など、Dart であらかじめ意味が定義された特別な単語のこと。
- キーワードは変数名などには使用できません。
- より多くのキーワードについては、[こちら](#)をご覧ください。

4. Dart 内蔵タイプ

- Dart にはさまざまなデータを扱うための組み込み型が用意されている：

◇ 数値 ([Numbers](#)) : `int`, `double`

`int`: 整数 (小数なし) & `double`: 浮動小数点数 (小数あり)

```
int x = 10;
double y = 3.14;
```

◇ 文字列 ([Strings](#)) : `String`

テキストを表すために使用する。

```
String greeting = 'Hello, World!';
```

- ◇ 真偽値 ([Booleans](#)) : bool
true または false の値を持つ.

```
bool isActive = true;
```

- ◇ レコード ([Records](#)) : (値 1, 値 2)
複数の値を 1 つにまとめる簡単な方法.

```
var record = (10, 'hello');
```

- ◇ 関数 ([Functions](#)) : Function
関数は処理を行い, 結果を返すことができる.

```
int add(int a, int b) {  
    return a + b;  
}
```

- ◇ リスト ([Lists](#)/ 配列) : List
複数の値を順序付きで保存する.

```
List<String> colors = ['red', 'blue', 'green'];
```

join() メソッドを使うと, リスト内の要素を 1 つの文字列に結合できる.

```
List<String> fruits = ['Apple', 'Banana', 'Cherry'];  
print(fruits.join(', ')); // Output: "Apple, Banana,  
Cherry"
```

- ◇ セット ([Sets](#)) : Set
重複のない値の集まる.

```
Set<int> uniqueNumbers = {1, 2, 3};
```

- ◇ マップ ([Maps](#)) : Map
キーと値のペアを保存する.

```
Map<String, int> scores = {'Alice': 90, 'Bob': 85};
```

5. Dart Generic

- ジェネリクスを使うことで, 異なるデータ型に対応できる柔軟なコードが書ける.
- 例えば, 整数だけを保持するリストを定義するには:

```
List<int> numbers = [1, 2, 3];
```

6. Dart のエラー処理

- プログラム実行中に問題が起きたときに,それを管理・処理する方法.
- Dartでは try,catch,finally を使う.

```
try {
  int result = 10 ~/ 0;
} catch (e) {
  print('Cannot divide by zero: $e');
}
```

7. Dart OOP

- Dart は OOP に対応しており,クラスを使ってオブジェクトを定義できる.
- オブジェクトはプロパティ (変数) とメソッド (関数) を持つ.

```
class Car {
  String make;
  String model;

  Car(this.make, this.model);

  void displayInfo() {
    print('Car: $make $model');
  }
}

void main() {
  var myCar = Car('Toyota', 'Corolla');
  myCar.displayInfo();
}
```

列挙型 (Enum)

- Enum (列挙型) は,特定の固定された値を表す特別な型.
- 特定の値しか許可したくないときに便利.

```
enum CarType { sedan, suv, truck }

void main() {
  CarType myCarType = CarType.sedan;

  if (myCarType == CarType.sedan) {
    print('Your car is a sedan.');

```

8. Dart Null Safety

- 変数が「null」になることによるエラーを防ぐ仕組み.
- Dartでは変数が null を取れるかどうかを明示する必要がある.

```
String? nullableString; // This can be null
```

```
String nonNullableString = 'Hello'; // This cannot be  
null
```


Flutter 入門

なぜ Flutter はウィジェットを使うのか：簡単な説明

a. ウィジェットとは？

- Flutter では, ボタン, テキスト, 画像など, すべてがウィジェットである.
- ウィジェットは UI (外観) とロジック (動作) を 1 つの場所で定義する.
- モジュール化, 再利用性, 高速開発を実現するためにウィジェットが使われる.

b. なぜ Flutter はウィジェットを使うのか？

- 宣言的アプローチ → 「何を表示したいか」を定義すれば, Flutter がレンダリングを担当する.
- UI とロジックの統合 → 定型コードが少なく, メンテナンスが簡単.
- シンプルなステート管理 → Stateless と Stateful を状況に応じて選択可能.
- 一貫したデザイン → Flutter のレンダリングエンジンにより, 異なるプラットフォームでも統一された UI を実現.

c. なぜこれは重要なのか？

- 同じウィジェットが UI と動作を定義 → デザインとロジックを分ける必要がない.
- 定型コードを削減 → 保守と共同開発が容易になる.
- Flutter の UI は完全にコンポーザブル → 小さなコンポーネントの組み合わせで構築可能.

d. ウィジェットのシンプルさ (例)

- テキストを表示するボタンを作る場合:

```
ElevatedButton(  
  child: Text('Click Me'), // The UI part  
  onPressed: () {           // The logic part  
    print('Button clicked!');  
  },  
);
```

e. このコードで何が起きているのか？

- Text('Click Me') → ボタンの見た目を定義.
- onPressed → ボタンがクリックされた時の動作を定義.
- UI とロジックが 1 つのウィジェットに統合されている.

f. Flutter の宣言的 UI

- 手動で UI を更新するのではなく, 最終的な UI 状態を定義する.
- ステートが更新されると, Flutter が自動で UI を再レンダリングする.

- 「どのように UI を更新するか」ではなく、「UI をどう見せるか」に集中できる.

Flutter は古い UI フレームワークとどう違うのか？

a. 従来の UI フレームワーク (Android, iOS)

- UI とロジックが異なるファイルに分かれている (例: XML で UI, Java/Kotlin でロジック) .
- 開発がより複雑 → 常にファイルを行き来する必要がある.
- 保守性が低い → UI を変更するには複数のファイルを修正する必要がある.

XML でボタンを定義:

```
<Button android:text="Click me" />
```

Java コードで動作を追加:

```
Button myButton = findViewById(R.id.myButton);
myButton.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        // Do something here
    }
});
```

b. Flutter の改善点

- ✓ UI とロジックがウィジェット内に統合されている.
- ✓ UI の見た目と動作を一箇所で管理できる.
- ✓ 開発が速く, コードがすっきりする → UI ファイルを別途管理する必要がない.

Flutter ではウィジェットだけを使用:

```
ElevatedButton(
  child: Text('Click Me'),
  onPressed: () {},
);
```

c. ウィジェットツリーの紹介: Flutter の UI 構造

- Flutter の UI は単純に並べられるのではなく, ウィジェットツリーという階層構造を形成する.
- 各ウィジェットは子ウィジェットを持つことができ, UI 全体を定義する.

- 従来の UI (Android XML, iOS Storyboard) はフラットな構造だったが,Flutter は柔軟で動的な UI 構成が可能.
 - アプリの状態が変化すると,Flutter はウィジェットツリーの必要な部分のみを再構築するため,UI 更新が効率的になる.
- d. ウィジェットは XML に似ているが,より強力
- どちらも構造化された UI 要素の定義を行う (XML はタグ,Flutter はウィジェット) .
 - どちらも属性/プロパティを使ってカスタマイズ可能 (例: `android:text="Click me"` vs. `Text('Click Me')`) .
 - ただし,Flutter のウィジェットの方が強力 → UI と動作を 1 つの場所で管理できる.

まとめ

- ウィジェット = Flutter の UI を構成する基本的な構成要素.
- 宣言的アプローチ = 手動で UI を更新する必要がない → 最終的な状態を定義すれば,Flutter がレンダリングを処理する.
- 単一のウィジェットに UI とロジックの両方を含む → メンテナンスが容易で,モジュール化が可能.
- ウィジェットのネストにより構成可能な UI を実現 → 複雑なインターフェースも,小さく再利用可能なパーツから構築できる.

基本概念：ウィジェットはどのように機能するのか？

Flutter で Key-Value 形式を理解する

a. Key-value 形式は何か？

- Flutter では、ウィジェットのプロパティはキーと値のペアで定義される。
- キー → プロパティを表す（例：style,color）。
- 値 → プロパティの動作を決める（例：Colors.blue,TextStyle）。
- 例として Text ウィジェットに使用：

```
Text(  
  'Hello, Flutter!',      // This is a value (text being  
displayed)  
  style: TextStyle(      // 'style' is a key  
    fontSize: 20,        // 'fontSize' is a key, and '20' is the  
value  
    color: Colors.blue,  // 'color' is a key, and 'Colors.blue'  
is the value  
  ),  
);
```

Hello, Flutter!

b. なぜ Key-value 形式を使用？

- ウィジェットのプロパティをカスタマイズ → UI の外観や動作を簡単に制御できる。
- デフォルト値を上書きできる → 色,サイズ,配置などを自由に変更可能。
- 可読性が向上 → 名前付きパラメータにより,コードが明確になる。

c. いつ key のデフォルト value を使用？

- 値を指定しない場合,Flutter はデフォルト値を適用する。
- 別の動作をさせたいときは,デフォルトを上書きする。
- 例：デフォルトのテキストカラーは黒 → color: Colors.blue に変更可能。
- 例として Text ウィジェットにデフォルトを上書きする：

```
Container(  
  width: 200,  
  height: 100,  
  color: Colors.blue, // This sets the background color to  
blue  
  child: Center(  
    child: Text(  

```

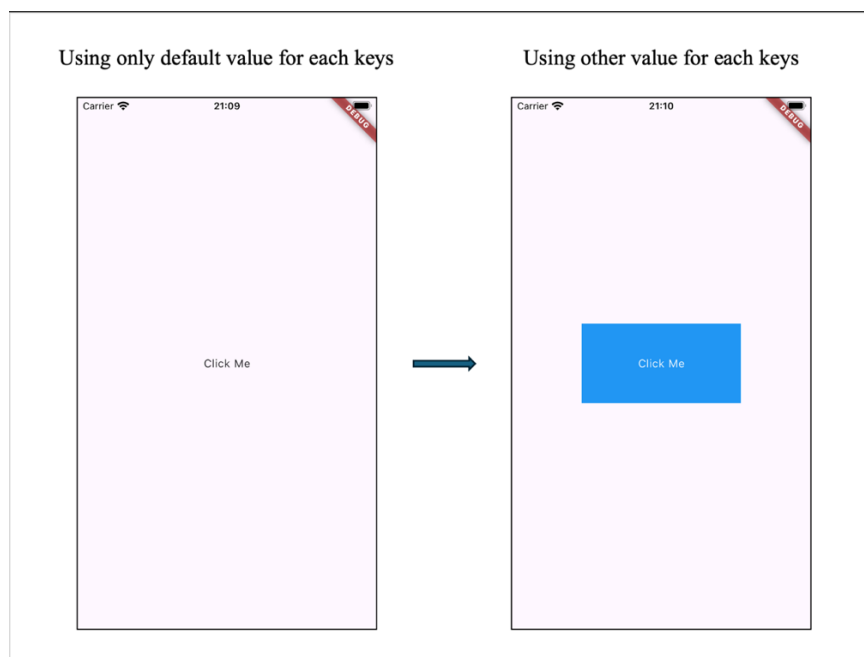
```

        'Click Me',
        style: TextStyle(color: Colors.white), // Text color
        is white
    ),
),
)

```

■ 説明：

- ◇ デフォルトでは,Container に背景色はない.
- ◇ `color: Colors.blue` を設定すると,背景色が青になる.
- ◇ `Text('Click Me')` は中央に配置され,白色で表示される.



d. Key-value 形式を Flutter で使用 vs JSON で使用

- Flutter と JSON はどちらもキーと値のペアを使うが,目的が異なる.
- Flutter → アプリの UI を定義する.

```

Text (
  "Hello, World!",
  textAlign: TextAlign.center,
)

```

- JSON → データの保存や転送を行う.

```

{
  "text": "Hello, World!",
  "textAlign": "center"
}

```

```
}
```

- 主な違い:

特徴	Flutter コード	JSON
目的	UI の構築と動作	データの保存と転送
実行方法	アプリ内で動作	データのやり取りに使用
例	<code>Text('Hello')</code>	<code>{ "text": "Hello" }</code>

Flutter でウィジェットがネストされる理由

a. ウィジェットネストとは？

- ウィジェットのネストとは、あるウィジェットを別のウィジェットの中に配置すること。
- これにより、UI 要素を組み合わせることでより複雑なデザインを作成できる。
- Flutter の UI は、ネストされたウィジェットの階層で構成されている。
- 例として `Text` ウィジェットが `Button` ウィジェットの中にネストされる：

```
ElevatedButton(  
  onPressed: () {  
    print('Button clicked!');  
  },  
  child: Text('Click Me'), // Text is inside the Button  
);
```

b. なぜウィジェットネストを使用？

- 関連する UI 要素をまとめる → コードの可読性が向上。
- デザインを柔軟にカスタマイズできる → スタイルやレイアウトの調整が簡単。
- 再利用がしやすい → 繰り返し使えるコンポーネントを作成可能。
- 例として `Text` ウィジェットが `Container` ウィジェットの中にネストされる：

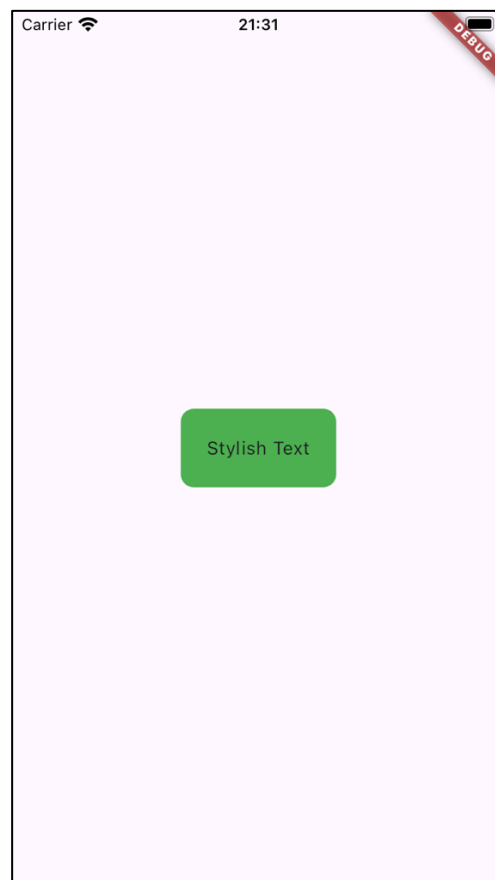
```
Container(  
  padding: EdgeInsets.all(10),  
  child: Text('This is a message'),  
);
```

c. どのようにネストはカスタムに役立つ

- ウィジェットを他のウィジェットの中に配置することで、外観を変更できる。
- 例えば、`Container` を使えばパディング、マージン、背景のスタイルを追加できる。

- 例として Text ウィジェットが Container ウィジェットの中にネストされる：

```
Container(  
  padding: EdgeInsets.all(20),  
  decoration: BoxDecoration(  
    color: Colors.green,  
    borderRadius: BorderRadius.circular(10),  
  ),  
  child: Text('Stylish Text'),  
);
```



なぜ Flutter はウィジェット内部で関数やプロパティを使うのか？

- a. なぜウィジェットで関数やプロパティが使われるのか？
 - Flutter では、ウィジェット内部で関数やプロパティを渡すことができる。
 - UI（見た目）とロジック（動作）を1つの場所で定義できるので便利。
 - 従来の UI フレームワークのように、UI とロジックを分ける必要がない。
- b. 例：onPressed でアクションを追加する（ボタンの例）
 - ボタンウィジェットの onPressed に関数を渡すことで、クリック時の動作を指定できる。

- 見た目（UI）と動作（ロジック）を一箇所にまとめることができる。
- 以下はコード例です：

```
ElevatedButton(
  onPressed: () {
    print('Button clicked!'); // This function runs when
the button is pressed
  },
  child: Text('Click Me'), // The text shown inside the
button
);
```

- 説明：
 - ◇ onPressed は関数を渡すためのプロパティ。
 - ◇ ボタンがクリックされると関数が実行される。
 - ◇ UI（テキスト）とロジック（クリック時の動作）が一緒に書かれている。
- c. コードを整理しやすくするために関数を分離する
 - 関数をウィジェットの外に定義し、参照することで、コードが整理され、保守性が向上する。
 - 関数が複雑な場合や、複数の場所で再利用する場合に便利。
 - 例えば、以下は関数を分けて使うコード例です：

```
void buttonClicked() {
  print('Button clicked!'); // This function runs when the
button is pressed
}

ElevatedButton(
  onPressed: buttonClicked, // Just reference the function
by its name
  child: Text('Click Me'), // The text shown inside the
button
);
```

- 関数を分離するメリット：
 - ◇ 再利用性 → 複数のボタンやイベントで関数を再利用できる。
 - ◇ 明確さ → UI コードがスッキリし、デザインに集中できる。
 - ◇ 保守性 → ボタンの動作を変更する場合、関数を 1 箇所変更するだけで済む。
- d. 例：プロパティを使ったスタイリング（Text ウィジェットの TextStyle）
 - プロパティを使うことで、ウィジェットの見た目を簡単にカスタマイズできる。
 - デザイン用のファイルを別に作らなくても、TextStyle などのプロパティでスタイルを定義できる。
 - コードが読みやすく、変更しやすくなる。

- 以下はコード例です：

```
Text(  
  'Hello, Flutter!',  
  style: TextStyle( // TextStyle customizes the appearance  
of the text  
    fontSize: 24,    // Makes the text larger  
    fontWeight: FontWeight.bold, // Makes the text bold  
    color: Colors.blue, // Changes the text color to blue  
  ),  
);
```

- 説明:

- ◇ style は TextStyle オブジェクトを受け取るプロパティ.
- ◇ fontSize,fontWeight,color はテキストの見た目を定義するキー.
- ◇ 別のスタイルシートを作らなくても,直接ウィジェット内でカスタマイズできる.

まとめ

- Flutter のキーと値のペアにより,カスタマイズと柔軟な設定が可能になる.
- デフォルト値を上書きすることで,UI の外観や動作を変更できる.
- Flutter の名前付きパラメータは,コードの可読性と保守性を向上させる.
- Flutter のキーと値の構造は JSON に似ているが,目的が異なる.
- ウィジェットのネストによって,UI 要素の構造を明確にできる.
- 関連するコンポーネントをひとまとめにすることで,コードが読みやすくなる.
- スタイル用ウィジェットでラップすることで,カスタマイズが容易になる.
- Flutter の UI はすべてネストされたウィジェットツリーで構築されている.
- ウィジェット内の関数は動作を定義する (例: onPressed) .
- 関数を分離することで,再利用性・明瞭性・保守性が向上する.
- ウィジェット内のプロパティは外観をカスタマイズする (例: Text の style) .
- Flutter は UI とロジックを一箇所に統合しているため,コードの管理がしやすい.

Flutter と Dart のプロジェクト構造の紹介

◇ 概要：Flutter アプリは何で構成されているのか？

- Flutter は Dart を使用し,オブジェクト指向プログラミング (OOP) でモジュール化されている.
- Flutter アプリはウィジェットのコレクションで構成される.
- すべての UI 要素 (テキスト,画像,ボタン,レイアウト) はウィジェットである.
- Flutter アプリはウィジェット階層構造を持ち,ウィジェットは他のウィジェットの中にネストできる (親子関係) .

◇ Flutter アプリの基本構造 (コード例)

- Flutter アプリは `runApp()` で開始する.
- `MaterialApp` はアプリの主要な構造を定義する.
- 各 UI 要素 (`Text`, `AppBar`, `Scaffold`) はウィジェットである.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Flutter App'),
        ),
        body: Center(
          child: Text('Hello, Flutter!'),
        ),
      ),
    );
  }
}
```

▪ 説明：

- `runApp(MyApp())` → アプリを起動する.
- `MaterialApp` → アプリ全体の構造を定義する.
- `Scaffold` → アプリのレイアウト (`AppBar`, `body` など) を提供する.
- `AppBar` → トップナビゲーションバーを定義する.
- `Text('Hello, Flutter!')` → アプリ内にテキストを表示する.

構造の分解（ステップごとの解説）

ステップ1：インポート 'package:flutter/material.dart' の説明

a. プログラミングにおけるインポートとは？

- プログラミングにおける「インポート」とは、他のファイルやライブラリからコードを取り込み、プロジェクトで使用することを意味する。
- 工具箱から必要なツールを借りて作業をするのと同じようなもの。
- Flutter では、事前に用意されたウィジェットや関数を使うためにライブラリのインポートが必要。

b. import 'package:flutter/material.dart'; はどういう意味か？

- これは、Flutter の UI コンポーネントを含むライブラリ。
- flutter → Flutter フレームワークのライブラリであることを示す。
- material → Google の「マテリアルデザイン」スタイルを使うことを意味する。
- dart → Dart プログラミング言語で書かれているライブラリ。
- この1行で、Flutter に用意された多くの UI コンポーネントにアクセスできるようになる。

```
import 'package:flutter/material.dart';
```

c. なぜこれを使うのか？

- ボタン、テキスト、レイアウトなどの UI コンポーネントを使用できるようにする。
- このインポートがなければ、Flutter の基本ウィジェットを使えない。
- 視覚的に魅力的なアプリを簡単に構築できるようになる。

d. material.dart によって提供される主要なウィジェット

ウィジェット	使用目的	例
Text	画面にテキストを表示する	<code>Text('Hello, Flutter!')</code>
ElevatedButton	インタラクティブなボタンを作成する	<code>ElevatedButton(onPressed: () {}, child: Text('Click Me'))</code>
Scaffold	基本的なレイアウト構造を提供する	<code>Scaffold(appBar: AppBar(title: Text('App')), body: Center(child: Text('Hello')))</code>
AppBar	上部のナビゲーションバーを作成する	<code>AppBar(title: Text('My App'))</code>

- これらのウィジェットはアプリの構造を作り、ユーザーとのインタラクションを定義する。
- `import 'package:flutter/material.dart';` がないと、これらの UI 要素を使用できない。

ステップ2：Flutter の main.dart の基本を理解する

a. Flutter における main() の役割

- Flutter アプリは main() から開始される (Java と同様) .
- ただし,main() の後に runApp() を呼び出す必要がある.
- runApp(MyApp()) は MyApp ウィジェットを表示し,UI を初期化する.
- これは,main.dart における基本的な Flutter コードの例:

```
void main() {  
  runApp(MyApp());  
}
```

- 説明:
 - ◇ main() → プログラムの開始点.
 - ◇ runApp(MyApp()) → Flutter アプリを起動し,MyApp() をロードする.

b. Flutter における runApp() の仕組み

- runApp() はアプリのルートウィジェットをロードする.
- MyApp() ウィジェットを表示し,UI 階層を構築する.
- Android の XML のような UI ファイルを使わず,すべてウィジェットで構築される.

c. 比較: Java の main() と Flutter の main()

- Java と Flutter はどちらも main() から実行が開始されるが,UI の扱いが異なる.
- Java (例: Android アプリ) → main() はロジックを開始するだけで,UI は XML で別途構築.

```
public class Main {  
  public static void main(String[] args) {  
    System.out.println("Hello World");  
  }  
}
```

- Flutter/Dart → main() の後,すぐにウィジェットで UI を構築.
- 主な違い:

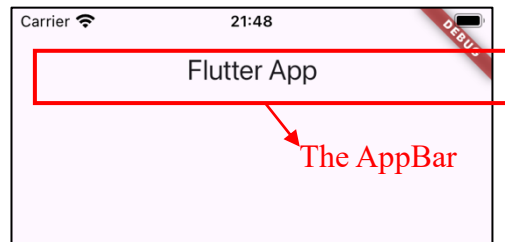
機能	Java (Android アプリ)	Flutter (Dart)
UI の処理	UI は XML で別途作成	UI は main.dart で直接構築
ワークフロー	main() でアプリを起動し, UI は別途処理	main() でアプリと UI を同時にセットアップ
管理のしやすさ	UI とロジックが分かれている	すべて 1 つのファイルで管理

ステップ3：MyAppとウィジェットツリーを理解する

a. MyAppとは何か？

- `runApp()` を呼び出した後、アプリは **MyApp** ウィジェットをロードする。
- **MyApp** は Flutter アプリのルートウィジェット。
- すべての Flutter アプリはウィジェットで構成され、UI を形成する。
- これは基本的な **MyApp** の実装例：

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: Text('My Flutter App'),  
        ),  
        body: Center(  
          child: Text('Hello, world!'),  
        ),  
      ),  
    );  
  }  
}
```



- 説明:
 - ◇ **MyApp** → アプリのエントリーポイントとして機能するカスタムウィジェット。
 - ◇ `StatelessWidget` → **MyApp** は状態を持たない（ステートレス）。
 - ◇ `build()` メソッド → UI の表示方法を定義する。
 - ◇ ウィジェットの階層構造（ウィジェットツリー）を返す。

b. ウィジェットツリーの仕組み

- Flutter の UI はツリー構造（ウィジェットツリー）で作られている。
- 各ウィジェットは子ウィジェットを持つことができ、階層を形成する。
- ウィジェットツリーはウィジェット同士の構造と関係性を定義する。
- これは **MyApp** 内のウィジェットツリーの例：

```
MaterialApp(  
  home: Scaffold(  

```

```

    appBar: AppBar(
      title: Text('My Flutter App'),
    ),
    body: Center(
      child: Text('Hello, world!'),
    ),
  ),
);

```

■ ツリー構造：

```

MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text ('My Flutter App')
│   └── Body
│       └── Center
│           └── Text ('Hello, world!')

```

■ 説明:

- ◇ MaterialApp → すべてを包むルートウィジェット。
- ◇ Scaffold → アプリの基本構造を提供する（AppBar, body など）。
- ◇ AppBar → アプリ上部のバー（タイトルやアクション用）。
- ◇ Text → 画面にテキストを表示するウィジェット。

c. ツリー内の各ウィジェットの理解

ウィジェット	使用目的	例
MaterialApp	テーマやナビゲーションを設定するルートウィジェット	MaterialApp(home: Scaffold(...))
Scaffold	画面の基本レイアウト（AppBar や body など）を提供する	Scaffold(appBar: ..., body: ...)
AppBar	上部ナビゲーションバーを表示する	AppBar(title: Text('My App'))
Center	子ウィジェットを画面の中央に配置する	Center(child: Text('Hello'))
Text	シンプルなテキストを表示する	Text('Hello, world!')

- Flutter の各ウィジェットは特定の機能を持ち,他のウィジェットの中にネストできる。
- このネスト構造により,UI を効率的に整理できる。

ステップ4：クラスとエクステンドを理解する

a. Flutter におけるクラスとは？

- クラスはオブジェクト（Flutter ではウィジェット）を作るための設計図.
- Flutter アプリは,クラスとして定義されたカスタムウィジェットで構築される.
- 各ウィジェットクラスは,UI の外観と動作を定義する.
- Flutter でウィジェットクラスを定義する一般的な方法はこちら：

```
class MyApp extends StatelessWidget {
```

▪ 説明:

- ◇ class MyApp → MyApp という名前のウィジェットを定義する.
- ◇ extends StatelessWidget → MyApp は StatelessWidget を継承しており,動的に変化しないウィジェット である.

b. Flutter における「extends」の意味とは？

- extends は,あるクラスを継承し,新しいクラスを作るためのキーワード.
- extends StatelessWidget を使うと,ウィジェットが変更されない（ステートレス）.
- ウィジェットは,親ウィジェットがリビルドしない限り更新されない.
- これは Stateless ウィジェットの完全な例：

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(title: Text('My Flutter App')),  
        body: Center(child: Text('Hello, Flutter!')),  
      ),  
    );  
  }  
}
```

▪ 説明:

- ◇ class MyApp extends StatelessWidget → MyApp はステートを持たないウィジェット.
- ◇ build() メソッド → ウィジェットの UI を定義する.
- ◇ MaterialApp → アプリのルートウィジェット.

c. super.key とは何か,なぜ使うのか？

- super.key は,StatelessWidget や StatefulWidget などの親クラスに key を渡すために使われる.
- Flutter がウィジェットツリー内のウィジェットを効率的に識別・更新するのに役立つ.

- 必須ではありませんが,ウィジェットの最適化や再利用性を高めるために 推奨される.

d. Java との比較

- Dart も Java も `extends` を使ってサブクラスを作る.
- Dartの方がシンプルで,Flutterのウィジェット設計に最適化されている.
- Javaでは通常,UIは別ファイルで定義されるが,Flutterではウィジェットクラス内に直接書く.

```
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

ステップ5 : `@override` を使ったメソッドのオーバーライド

a. Flutterにおける`@override`とは？

- `@override` は,子クラスのメソッドが親クラスのメソッドを置き換えるときに使う.
- Flutterでは,`@override` は `StatelessWidget` や `StatefulWidget` から継承された `build()` によく使用される.
- 親クラスのメソッドを正しくオーバーライドしていることを保証する.
- `build()`メソッドをオーバーライドする例：

```
@override
Widget build(BuildContext context) {
```

- 説明:
 - ◇ `@override` → この `build()` メソッドが親クラスの `build()` を置き換えていることをFlutterに伝える.
 - ◇ `build(BuildContext context)` → UIの表示方法を定義する.
 - ◇ `@override` がなくても Dartは動作するが,コードの明確性を向上させ,ミスを防ぐのに役立つ.

b. なぜ`@override`が重要なのか？

- メソッドが正しくオーバーライドされていることを保証する.
- コードを読みやすくする → このメソッドが親クラスのメソッドを置き換えていることを明示的に示す.
- 意図しないミスを防ぐ → `@override` がないと,メソッド名を間違えたときに Dartがエラーを検出しない可能性がある.

c. 比較：Java と Flutter における`@override`

- Java も Flutter (Dart) も `@override` を使って,メソッドのオーバーライドを明示する.

- メソッドのシグネチャが一致しない場合のミスを防ぐ.
- Java では明示的なメソッドシグネチャが必要だが,Dart はより柔軟.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- 説明:
 - ◇ @Override → Child の build() が Parent の build() を正しくオーバーライドしていることを保証する.
 - ◇ メソッド名が間違っていた場合,Java はエラーを出す.
- 主な違い:

機能	Java	Flutter
@override の用途	OOP メソッドのオーバーライド	Flutter ウィジェットのオーバーライド
エラー防止	誤ったメソッドシグネチャを防ぐ	タイプミスの検出を助ける
柔軟性	厳格なメソッド定義	より柔軟だが,@override で明確

ステップ6：build() メソッドとBuildContextの探索

a. build()メソッドとは？

- build() メソッドは,ウィジェットが画面にどのように表示されるかを定義する.
- “ウィジェットツリー” (UIを構成するウィジェットの階層構造) を返す.
- UIが更新される必要があるときに,build() メソッドが再び呼び出される.

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                body: Center(
                    child: Text('Hello, Flutter!'),
                ),
            ),
        );
    }
}
```

```
}
```

- 説明:
 - ◇ build() は MaterialApp ウィジェットを返し,アプリの UI 全体を定義する.
 - ◇ MaterialApp の中に Scaffold,Center,Text などのウィジェットが含まれる.
 - ◇ この構造がウィジェットツリーを形成し,UI のレイアウトを決める.

b. BuildContext とは？

- BuildContext は,ウィジェットがウィジェットツリーのどこにあるかを示す情報を提供する.
- 継承されたウィジェットにアクセスしたり,ナビゲーションを管理したりできる.
- build() メソッドの必須パラメータである.

c. build()が UI を更新し続ける仕組み

- Flutter は UI の変化を検出すると build() を再び呼び出す.
- UI の最新状態を確実に表示するために build() を利用する.
- Flutter は宣言的 UI モデルを採用しており,手動で UI を変更するのではなく,再構築する.

d. 比較：Java における UI の構築方法との違い

- Java (Android) では,通常,UI の更新は View オブジェクトの変更によって行われる.
- Flutter では,UI は宣言的に構築され,build() を使って再構築される.
- Flutter のアプローチにより,UI の更新が構造化され,予測しやすくなる.

```
@Override
protected View build(Context context) {
    TextView textView = new TextView(context);
    textView.setText("Hello, Android!");
    return textView;
}
```

- 主な違い

機能	Java (Android)	Flutter (Dart)
UI の構造	XML や Java コードで View を作成	build() メソッドでウィジェットツリーを返す
状態管理	View を手動で更新	状態が変わると UI が自動更新
柔軟性	View 階層を変更する必要がある	build() によりシンプルに管理

ステップ 7：Flutter における return の役割

a. build()における return の役割とは？

- build() メソッド内で return を使って,ウィジェットが表示する内容を指定する.

- レンダリングのためにウィジェットを Flutter に送り返す.
- Flutter は return を使って UI 要素を構築し,動的に表示する.

b. return で返されるウィジェットの理解

ウィジェット	使用目的	例
MaterialApp	ナビゲーション, テーマ, ローカライゼーションを提供するルートウィジェット	<code>return MaterialApp(...)</code>
Scaffold	AppBar や body などを含むメイン画面の構造を定義する	<code>return Scaffold(...)</code>
AppBar	アプリのタイトルや操作を表示する上部ナビゲーションバー	<code>AppBar(title: Text('My App'))</code>
Text	画面にシンプルなテキストを表示する	<code>Text('Hello, world!')</code>

- ポイント:
 - ◇ Flutter では,build() 内で return を使ってウィジェットを返さないと画面に表示されない.
 - ◇ return がないと,Flutter は何を表示すべきか判断できない.

c. 比較: Java における return の使われ方

- Java も Flutter (Dart) も,return を使って作成したオブジェクトを返す.
- Java では return は View や UI コンポーネントを返すことが多い.
- Flutter では,return はよりシンプルでウィジェット中心の設計になっている.
- 主な違い:

機能	Java (Android)	Flutter (Dart)
戻り値の型	View (TextView,Button など) を返す	Widget (Text,Scaffold など) を返す
UI の構造	UI は通常 XML で管理	Dart 内でウィジェットを使って UI を構築
柔軟性	View の階層を変更する必要がある	build() メソッド内で簡単に管理可能

ステップ8: Flutter の MaterialApp とは?

a. MaterialApp とは?

- MaterialApp は,Google の Material Design に準拠した Flutter アプリの基盤となる.
- アプリの基本構造とビジュアルスタイルを提供する.
- テーマ,ナビゲーション,デフォルトの動作を設定し,スムーズなユーザー体験を実現する.

- MaterialApp の主な機能:
 - ◇ ビジュアルスタイルの定義 (テーマ,色,フォント) .
 - ◇ 異なる画面間のナビゲーション管理.
 - ◇ デフォルトの動作を提供し,開発を簡単にする.

b. MaterialApp の基本的な例

- Flutter のコード例:

```
MaterialApp(
  home: Scaffold( // The main layout for the screen
    appBar: AppBar( // The top bar with a title
      title: Text('Welcome to Flutter!'),
    ),
    body: Center( // The main content in the middle of
the screen
      child: Text('Hello, Flutter!'),
    ),
  ),
);
```

- MaterialApp の構造の分解・解説

プロパティ	機能	例
home	アプリ起動時に最初に表示される画面を定義する	home: Scaffold(...)
Scaffold	AppBar や body を含む主要なレイアウトを提供する	Scaffold(appBar: ..., body: ...)
AppBar	タイトル付きの上部ナビゲーションバーを表示する	AppBar(title: Text('Welcome'))
body	画面のメインコンテンツ領域を定義する	body: Center(child: Text('Hello, Flutter!'))

- 説明:
 - ◇ home → アプリ起動時に最初に表示される画面.
 - ◇ Scaffold → 画面の基本的なレイアウトを提供する (AppBar,body を含む) .
 - ◇ AppBar → タイトルやボタンを配置できるトップバー.
 - ◇ body → メインコンテンツ (テキスト,画像,ボタンなど) .

c. なぜ MaterialApp が重要なのか?

- アプリの基本構造を提供し,ゼロから作る手間を省く.
- Google の Material Design に準拠し,モダンで統一感のあるデザインを提供.

- 異なる画面間のナビゲーションを簡単に整理できる.
- MaterialApp の主なメリット:
 - ◇ すぐに使える構造 → 開発時間を短縮.
 - ◇ Material Design 準拠 → UI の一貫性を確保.
 - ◇ 簡単なナビゲーション管理 → 複数画面の管理が容易.
- d. 比較: Java におけるアプリのセットアップ方法
 - Java (Android 開発) では,UI コンポーネントは通常 XML ファイルで作成される.
 - Flutter では,MaterialApp とウィジェットを使って UI を宣言的に構築する.
 - Flutter のアプローチは,コードの複雑さを減らし,保守性を向上させる.

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

- 主な違い:

機能	Java (Android)	Flutter (Dart)
UI の構造	XML ファイルでレイアウトを定義	Dart コード内で直接 UI を作成
ナビゲーション設定	Intent や Fragment を使用	MaterialApp で簡単に管理
複雑さ	コードが長くなりやすく, 手動更新が必要	シンプルで宣言的な UI モデル

ステップ 9 : Flutter の Scaffold とは？

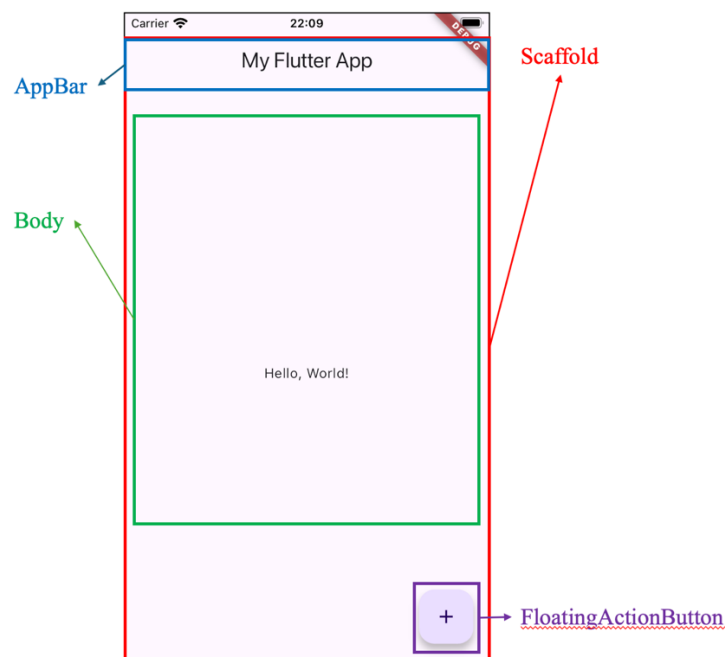
- a. Scaffold とは？
 - Scaffold は,Flutter アプリの画面の基本構造 (スケルトン) を提供する.
 - AppBar,body,FloatingActionButton などの主要な UI 要素を整理する.
 - UI レイアウトを簡素化し,アプリのインターフェースを構築しやすくする.
 - Scaffold の主要な UI コンポーネント:
 - ◇ AppBar → タイトルやナビゲーションボタンを表示するトップバー.
 - ◇ body → 画面のメインコンテンツ領域.
 - ◇ floatingActionButton → すぐにアクションを実行できるフローティングボタン.
- b. Scaffold の基本的な例
 - Flutter のコード例:

```
Scaffold(
  appBar: AppBar(
```

```

    title: Text('My Flutter App'),
  ),
  body: Center(
    child: Text('Hello, World!'),
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () {
      print('Button Pressed!');
    },
    child: Icon(Icons.add),
  ),
);

```



■ Scaffold の主要な構成要素

◇ appBar - 上部のナビゲーションバー

- ナビゲーションとタイトルを表示するトップバーを作成する.
- タイトル (Text) やアクションボタンを含むことが多い.
- appBar プロパティ内で AppBar ウィジェットを使用する.

◇ body - メインコンテンツエリア

- 画面の中で最も大きな部分で,ほとんどのウィジェット (テキスト,画像,ボタンなど) を配置する.
- Center, Column, Row などのウィジェットを使ってレイアウトを整理する必要がある.
- Text, ListView, GridView などのウィジェットを含むことが多い.

◇ floatingActionButton - クイックアクションボタン

- コンテンツの上に「浮く」ボタンで,クイックアクションに使用される.
- アイコンや他のウィジェットを内部に配置する必要がある.

- アイテムの追加やコンテンツのリフレッシュなどのアクションに使われる。
- c. なぜ Scaffold が重要なのか？
 - 構造化されたレイアウトを提供し,画面を作成しやすくする.
 - アプリの異なる部分で一貫した UI 体験を実現する.
 - MaterialApp と統合し,スムーズなナビゲーションとテーマ管理を可能にする.
 - Scaffold のメリット:
 - ◇ 適切な構造を提供 → UI デザインを簡素化する.
 - ◇ ユーザー体験を向上 → モダンでレスポンスなレイアウトを実現.
 - ◇ Material Design と統合 → UI の一貫性を確保する.

まとめ

- Flutter アプリは main() と runApp() から始まり,MyApp ウィジェットを読み込む.
- MyApp は StatelessWidget を継承し,ルートウィジェットである MaterialApp を返す.
- MaterialApp の中で Scaffold が使用され,画面レイアウトが構築される.
- build() メソッドはウィジェットツリーを構築し,return によって UI が定義される.
- AppBar,body,floatingActionButton などの主要なウィジェットは Scaffold の中に配置される.
- Flutter のウィジェットシステムは階層構造になっており,すべてがウィジェットの中に構築されている.

Stateless vs Stateful ウィジェット

StatelessWidget と StatefulWidget を理解する

a. StatelessWidget と StatefulWidget とは？

- Flutter には主に 2 種類のウィジェットがある: StatelessWidget と StatefulWidget.
- どちらも UI を構築するために使用されるが、動作が異なる.
- Stateless widget は構築後に変更されず, Stateful widget は動的に変化する.

b. StatelessWidget とは？

- StatelessWidget は、一度作成されたら変更されない.
- Flutter が明示的に再構築しない限り、状態はそのまま.
- 静的なコンテンツ (テキスト、アイコン、画像など) の表示に最適.

```
class TitleWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text('Welcome to My App');  
  }  
}
```

- 説明:

- ◇ ウィジェットは常に「Welcome to My App」と表示される.
- ◇ アプリ内で何が起こっても、このテキストは変わらない.

c. StatefulWidget とは？

- StatefulWidget は時間とともに変化する.
- UI 要素が動的に更新される必要がある場合に使用.
- よく使われる場面: ボタン、フォーム、ユーザーインタラクション.

```
class ButtonWidget extends StatefulWidget {  
  @override  
  _ButtonWidgetState createState() =>  
    _ButtonWidgetState();  
}  
  
class _ButtonWidgetState extends State<ButtonWidget> {  
  String buttonText = 'Press me';  
  
  void updateText() {  
    setState(() {  
      buttonText = 'You pressed the button!';  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {
```



```

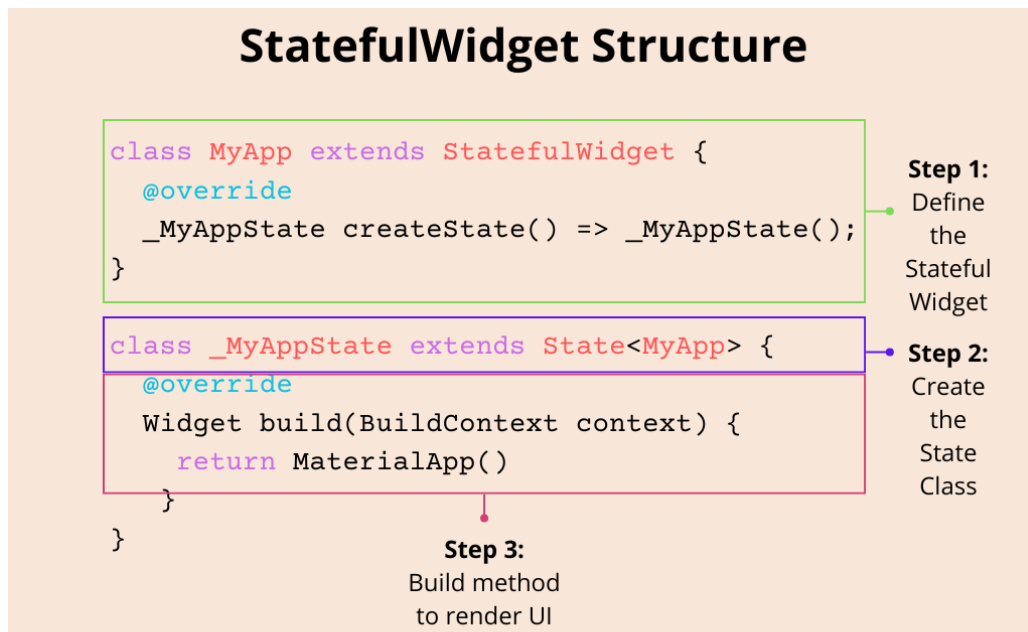
        return ElevatedButton(
            onPressed: updateText,
            child: Text(buttonText),
        );
    }
}

```

■ 説明:

- ◇ `_ButtonWidgetState` は, `ButtonWidget` の状態を 管理するクラス です.
- ◇ `State<ButtonWidget>` を継承しており, `ButtonWidget` というウィジェットに対応している.
- ◇ `build()` メソッドは オーバーライドされて, ウィジェットの見た目を定義する.
- ◇ `setState()` を呼び出すことで, UI が更新される.

d. `StatefulWidget` の構造



■ 構造の説明:

ステップ	起こること	目的	例
1	特別なウィジェット型を継承するクラスを定義する	状態を持つ動作を可能にする	<pre>class ButtonWidget extends StatefulWidget</pre>
2	ロジックコントローラーと接続するメソッドを提供する	状態を保持するオブジェクトを返す	<pre>@override _ButtonWidgetState createState() => _ButtonWidgetState();</pre>

3	変更を管理するための2つ目のクラスを作成する	ウィジェットがどのように反応するかを決定する	<pre>class _ButtonWidgetState extends State<ButtonWidget> {}</pre>
4	UIの更新を行う特別なメソッドを使う	Flutterにウィジェットの再構築を通知する	<pre>setState(() { buttonText = 'Updated!'; });</pre>

- ◇ ウィジェットクラスの中には、スーパークラスから再定義されたメソッドが存在する。
- ◇ そのメソッドの戻り値は、ロジックを処理する別のクラスのインスタンス。
- ◇ ロジック処理クラスは常に、ウィジェットを型引数として受け取る別のクラスを継承している。
- ◇ ビルド構造は、そのロジック処理クラスの中で定義され、ウィジェットを構築して返すメソッドによって記述される。

e. StatelessWidget と StatefulWidget の違い

■ 比較表

特徴	StatelessWidget	StatefulWidget
状態が変わるか?	いいえ	はい
状態管理	内部状態なし	内部状態あり
パフォーマンス	より効率的	効率が悪い
例	静的テキスト, アイコン, 画像	インタラクティブなボタン, フォーム
ウィジェットクラス	<pre>class MyWidget extends StatelessWidget</pre>	<pre>class MyWidget extends StatefulWidget</pre>
構築方法	build() のみ必要	createState() + State クラスが必要

f. 比較: Java (Android 開発) との違い

- Java (Android) では、UI コンポーネントの更新に View と Listener を使用する。

```
TextView textView = findViewById(R.id.myTextView);
textView.setText("Welcome to My App");
```

```
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        button.setText("You pressed the button!");
    }
});
```

- Flutter では、宣言的に setState() を使って UI を更新する。

Flutter で setState を理解する

a. setState とは？

- setState は Flutter にウィジェットの状態が変わったことを通知し,UI の再構築を引き起こす.
- 状態を変更する関数を受け取る.
- ボタンクリック,フォーム入力,アニメーションなどのユーザー操作を処理するのに使用される.
- これはカウンターアプリの例:

```
class CounterWidget extends StatefulWidget {  
  @override  
  _CounterWidgetState createState() =>  
    _CounterWidgetState();  
}  
  
class _CounterWidgetState extends State<CounterWidget> {  
  int counter = 0;  
  
  void incrementCounter() {  
    setState(() {  
      counter++; // Update the counter  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Text('Counter: $counter'),  
        ElevatedButton(  
          onPressed: incrementCounter,  
          child: Text('Increment'),  
        ),  
      ],  
    );  
  }  
}
```

- 説明:
 - ◇ カウンターは最初 0 に設定されている.
 - ◇ ボタンが押されると,incrementCounter() が呼ばれる.
 - ◇ setState() 内で counter が更新され,UI が再構築される.
 - ◇ 新しいカウンターの値が画面に表示される.

b. setState と Java (Android 開発) の比較

- 比較表

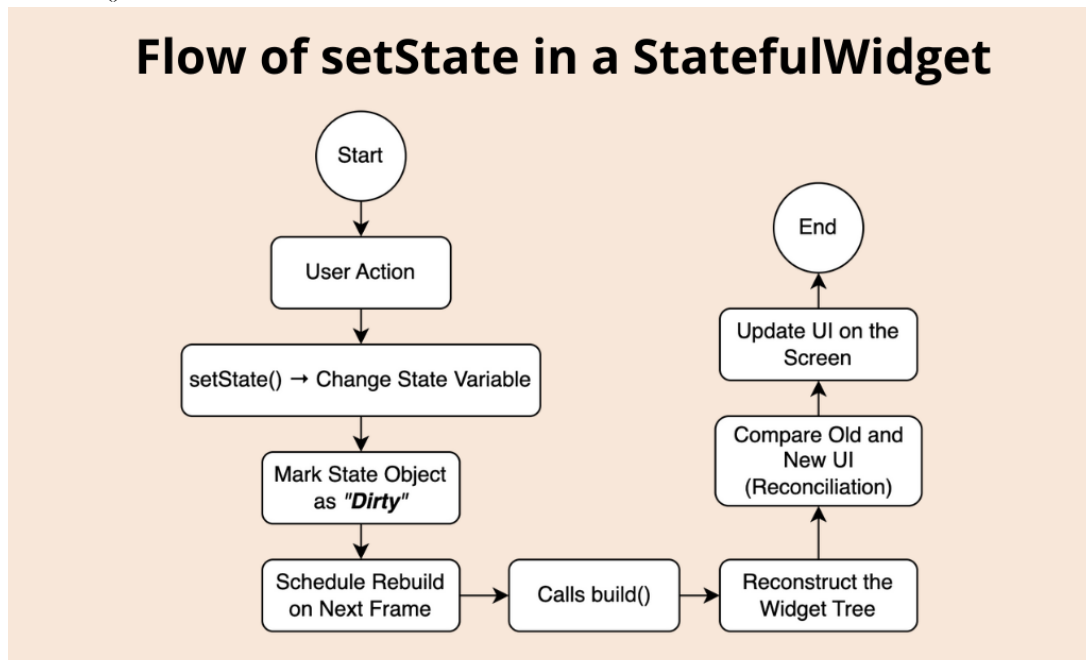
特徴	Flutter (setState)	Java (Android)
----	--------------------	----------------

状態更新の方法	setState()	イベントリスナー内で UI を直接変更
UI の更新	自動で再構築される	手動でビューを更新する必要がある
再構築の効率	必要な部分のみ更新	明示的にビューを変更する必要がある
例	ボタンのクリックでテキストを変更	TextView を手動で更新

■ 主な違い:

- ◇ Java では TextView を手動で更新する必要がある.
- ◇ Flutter の setState() は自動的に UI を更新する.
- ◇ Flutter は宣言的アプローチ,Java は命令的アプローチを使用する.

c. setState()の流れ



1. 状態の変更がトリガーされる
setState() { ... } が呼び出されると、ウィジェットの状態にある値が変更される。
2. ウィジェットが「ダーティ」とマークされる
Flutter は State オブジェクトを「ダーティ (dirty)」としてマークする.これは、そのウィジェットの状態が変化し,UI を更新する必要があることを意味する。
3. 再ビルドのスケジュールが設定される
Flutter はウィジェットの再ビルドをスケジュールする.ただし,すぐに画面を更新するのではなく,次のフレームまで待って,UI の更新を効率よくまとめて処理する。
4. build()メソッドが再度呼び出される
State オブジェクト内の build()メソッドがもう一度呼び出される.Flutter は,変更された状態に依存する部分だけのウィジェットツリーを再構築する。

5. UIが更新される

Flutter は,古いウィジェットツリーと新しいウィジェットツリーを比較する
(このプロセスはウィジェットの照合と呼ばれます) .

6. 変更された部分のみが画面上で更新される.

まとめ

- StatelessWidget は静的な UI 向け,StatefulWidget はインタラクティブな UI 向け.
- StatefulWidget では,UI を更新するために State クラスと setState()の使用が必要.
- Flutter は宣言的 (declarative) に状態を管理します.Java のように View を手動で更新する必要はありません.
- setState()は StatefulWidget の状態を更新し,UI の再ビルドをトリガーする.
- Flutter は,必要な部分のみを効率よく更新することで,パフォーマンスを保つ.
- Java (Android) のような手動 UI 更新とは異なり,Flutter は宣言的なステート管理を採用しており,よりシンプルで安全.
- setState()は以下のような構造的な流れで処理される :
- 状態変更 → ダーティとしてマーク → 再ビルドをスケジュール → build()呼び出し → UI 更新

ウィジェットの階層構造と発展的な概念

Flutter のウィジェット階層を理解する

a. ウィジェット階層構造とは？

- Flutter の UI は,ウィジェットを入れ子にしたツリー構造で構築される.
- この構造は「ウィジェットツリー」と呼ばれ,UI を効率的に整理する.
- Flutter では,各ウィジェットが別のウィジェットの中にネストされ,階層を形成する.

b. なぜウィジェット階層構造を使うのか？

- メリット:

特徴	説明
整理された構造	UI 構造が明確になり, 開発がしやすくなる.
再利用可能	ウィジェットをアプリ全体で再利用できる.
管理しやすい	1つのウィジェットを変更しても, アプリ全体に影響を与えない.
柔軟なカスタマイズ	ネストを利用して簡単にスタイルを適用できる.
パフォーマンス最適化	変更が必要な部分だけ再構築されるので, 効率が良い.

c. 例：基本的なウィジェット階層構造

- 例の Flutter コード:

```
return MaterialApp(  
  home: Scaffold(  
    appBar: AppBar(  
      title: Text('Hello Flutter!'),  
    ),  
    body: Center(  
      child: Text('Welcome to my app!'),  
    ),  
  ),  
);
```

- 階層の説明:

レベル	ウィジェット	親ウィジェット	説明
1 (Top)	MaterialApp	-	アプリのルートウィジェット
2	Scaffold	MaterialApp	アプリの基本構造
3	AppBar	Scaffold	ナビゲーションバー
3	Center	Scaffold	中央に配置するウィジェット
4 (Bottom)	Text	Column	テキストを表示

d. ウィジェット階層における再利用可能なコンポーネント

- ウィジェットを階層化することで,UI を再利用可能なコンポーネントに分割できる.
- 例えば,カスタムボタンを作成すれば,アプリ内のさまざまな場所で使える.

```
class CustomButton extends StatelessWidget {  
  final String text;  
  CustomButton(this.text);  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(  
      onPressed: () {},  
      child: Text(text),  
    );  
  }  
}
```

- メリット:
 - ◇ 異なるテキストでどこでも再利用可能.
 - ◇ 重複したボタンコードを減らせる.

e. 階層内のウィジェットの管理とカスタマイズ

- ウィジェット階層のおかげで,UI のカスタマイズが簡単になる.
- 例えば,テキストにパディングや背景色を追加する場合,Container と Padding を使えば OK.

```
Container(  
  padding: EdgeInsets.all(8.0),  
  color: Colors.blueAccent,  
  child: Text('Styled Text'),  
);
```

- 説明:
 - ◇ Container で背景色を追加.
 - ◇ Padding で余白を追加.

f. ウィジェット階層によるパフォーマンス上の利点

- Flutter は必要なウィジェットのみを再構築し,無駄なリビルドを防ぐ.
- 入れ子のウィジェットが多くても,スムーズに動作する.
- Flutter の UI 更新処理:

アクション	処理内容
ウィジェットが変更される	変更されたウィジェットとその子のみ再構築される.
親ウィジェットは変更なし	UI の影響を受けない部分は更新されない.

Flutter は不要な更新をスキップ	パフォーマンスを向上させ, 処理を最適化する.
----------------------------	-------------------------

g. Flutter のウィジェット階層 vs Java (Android 開発)

- 主要な違い:

特徴	Flutter (Dart)	Android (Java)
UI の定義	レイアウトとロジックが Dart 内で完結	UI は XML, ロジックは Java/Kotlin
ウィジェット構造	すべてがウィジェット (ネスト構造)	ビューは別オブジェクトとして ID で管理
状態管理	setState() で宣言的に管理	リスナーで手動更新

第一級オブジェクトとしての関数と引数としての使用

a. 「第一級オブジェクト」とはどういう意味か?

- Dart では,関数は「第一級オブジェクト」として扱われ,他の変数と同じように扱うことができる.
- 関数を変数に代入したり,引数として渡したり,別の関数から返したりできる.
- これにより,コードの柔軟性と再利用性が向上する.

b. 第一級クラス関数の主な使い方

- Dart の関数の主な使い方

特徴	説明	例
変数に代入	関数を変数として保存できる.	<code>var fn = sayHello;</code>
引数として渡す	関数を別の関数に渡せる.	<code>executeFunction(sayHello);</code>
関数を返す	関数から別の関数を返すことができる.	<code>Function getFunction() { return sayHello; }</code>

c. 例: 関数を引数として渡す

```
void sayHello() {
    print('Hello!');
}

void executeFunction(void Function() fn) {
    fn(); // Call the function passed as an argument
}

void main() {
    executeFunction(sayHello); // Pass the sayHello function
}
```

- 説明:

- ◇ sayHello() 関数を executeFunction() に渡す.
- ◇ executeFunction() の中で,渡された関数 fn() を実行.
- ◇ これにより,関数を柔軟に実行できる.

d. 例：引数付きの関数

```
void showMessage(String message) {
    print(message);
}

void performAction(void Function(String) action, String
message) {
    action(message); // Call the action with the message
}

void main() {
    performAction(showMessage, 'Button was clicked!'); //
Pass the showMessage function
}
```

- 説明:
 - ◇ showMessage(String message) はメッセージを表示する関数.
 - ◇ performAction() は関数 (action) と message を受け取る.
 - ◇ performAction() の中で,渡された関数がメッセージと共に実行される.

e. なぜ Flutter においてこれは有用なのか？

- 関数を引数として渡すことは,Flutter でのユーザーインタラクション処理に特に便利.
- 例えば,Flutter のボタン (ElevatedButton) は,onPressed プロパティで関数を受け取る.

f. 応用例：関数を戻り値として返す

```
Function createMultiplier(int factor) {
    return (int number) => number * factor;
}

void main() {
    var doubleIt = createMultiplier(2); // Returns a function
that multiplies by 2
    print(doubleIt(5)); // Output: 10
}
```

- 説明:
 - ◇ createMultiplier(int factor) は factor で掛け算する関数を返す.
 - ◇ createMultiplier(2) を呼び出すと,2 倍する関数が返される.
 - ◇ doubleIt(5) を実行すると 10 になる.

g. Dart と Java の関数の違い

- 主な違い:

特徴	Dart (Flutter)	Java (Android)
関数の扱い	関数は第一級オブジェクト	関数はクラス内のメソッド
関数の引数渡し	簡単に渡せる	インターフェースや匿名クラスが必要
関数の返却	関数を返せる	インターフェースやラムダ式を利用

三項条件操作

a. 三項演算子（条件演算子）とは？

- 三項条件とは,if-else 文を 1 行で書くためのショートカットである.
- 条件をチェックし,true または false に応じて処理を決定する.
- シンプルな条件を扱う場合,コードを短く,読みやすくできる.

b. 基本構文

```
condition ? valueIfTrue : valueIfFalse;
```

- 構文の説明:

- ◇ condition → チェックする条件 (true または false).
- ◇ ? → condition が true の場合,この後の値が返される.
- ◇ : → condition が false の場合,コロンの後の値が返される.

c. 例: if-else 文を三項演算子に書き換える

Regular if-else (通常の if-else)	Ternary Condition (三項条件演算子)
<pre>if (isLoggedIn) { return Text('Welcome back!'); } else { return Text('Please log in!'); }</pre>	<pre>Text(isLoggedIn ? 'Welcome back!' : 'Please log in!');</pre>

- 説明:

- ◇ if-else を複数行で書く必要がなくなる.
- ◇ isLoggedIn に基づいて適切な値を直接返す.

d. 三項演算子を使うべき場面は？

- 三項条件を使うべきとき:
 - ◇ 条件がシンプルで,2つの値のどちらかを選ぶ場合.
 - ◇ 短い表現で素早く判断したい場合.
 - ◇ UI ウィジェット (Text, Button など) でコードを簡潔にしたい場合.

e. 通常の if-else を使うべき場面は？

- 通常の if-else を使うべきとき:
 - ◇ 条件が複雑で、複数のチェックが必要な場合.
 - ◇ 結果を返す前に追加の処理が必要な場合.
 - ◇ 三項条件を使うと、コードが読みづらくなる場合.
- こちらは,if-else 文の方が三項演算子より適している例:

```
if (userAge >= 18) {  
  print('You are an adult.');} else if (userAge > 13) {  
  print('You are a teenager.');} else {  
  print('You are a child.');}
```

- 説明:
 - ◇ 三項条件は true または false の 2 つの選択肢に適している.
 - ◇ 複数の条件がある場合 (if-else if-else),通常の if-else のほうが明確.

f. Flutter の UI ウィジェットにおける三項演算子の使用例

- Flutter UI での使用例:
 - ◇ 条件に応じて異なる UI を表示.
 - ◇ Text, Container, Button などのウィジェットでよく使われる.
 - ◇ UI コードをシンプルにできる.
- こちらは,条件に応じてテキストを変更する例

```
Text(user.isLoggedIn ? 'Welcome, ${user.name}!' : 'Please  
log in.');
```

- 説明:
 - ◇ user.isLoggedIn が true なら "Welcome, {name}!" を表示.
 - ◇ それ以外は "Please log in." を表示.

まとめ

- Flutter の UI は,ウィジェットの階層（ウィジェットツリー）によって構築されている.
- この階層構造により,整理・再利用・カスタマイズ・パフォーマンスが向上する.
- Flutter は変更されたウィジェットのみを効率的に更新し,無駄な再描画を減らす.
- Java (Android) とは異なり,Flutter ではレイアウトとロジックを一箇所で管理でき,UI の管理が簡単になる.
- Dart では関数が第一級オブジェクトとして扱われ,変数として保持・渡す・返すことができる.

- これにより,ユーザー操作や再利用可能なコンポーネントの処理が簡単になる.
- Java と比べて,Dart はより簡潔かつ柔軟に関数を扱うことができる.
- 三項条件演算子は if-else の省略形で,コードを簡潔にできる.
- 特に Text や Button などの UI 要素で簡単な条件を処理するときに便利.
- 複雑な条件には,可読性を保つために通常の if-else 文を使う方が良い.
- 三項条件をネストしすぎるとコードが読みにくくなるので注意すること.

よく使われるウィジェット

Flutter の AppBar ウィジェット

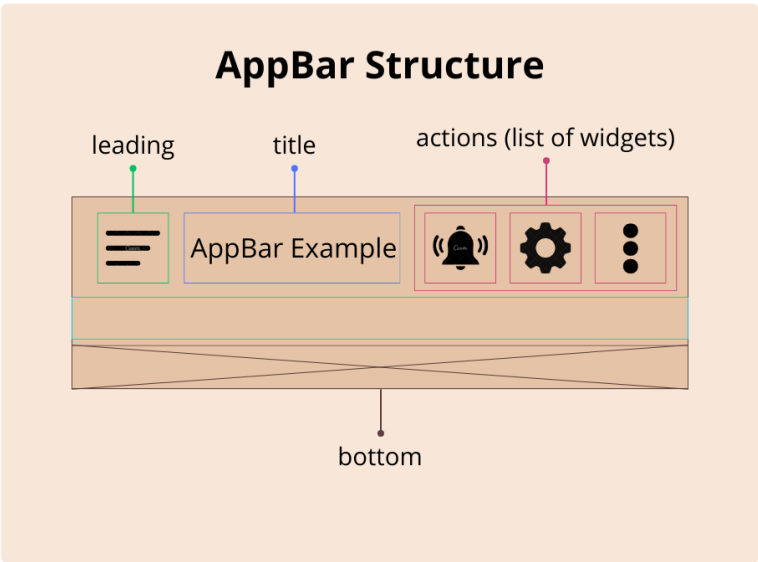
a. AppBar（アプリバー）とは？

- AppBar は、画面の上部に配置されるツールバーである。
- コンテンツを整理し、アプリの一貫した構造を提供する。
- タイトル、ナビゲーションアイコン、アクションボタンの表示によく使われる。

b. AppBar の主な構成要素

- AppBar の主なパーツ:

パーツ	説明	使用例
Leading	通常はナビゲーションアイコン (例: メニュー, 戻るボタン)	leading: Icon(Icons.menu)
Title	バーの中央に表示されるテキスト またはウィジェット	title: Text('My AppBar')
Action	右側に配置されたアイコンやボ タン	actions: [IconButton(icon: Icon(Icons.search))]
Bottom	AppBar の下部に追加されるウィ ジェット (例: タブバー)	bottom: PreferredSize(...)



c. 例：Flutter での AppBar の使用例コード

- AppBar の基本的な実装:

```
AppBar(  
  leading: Icon(Icons.menu), // Icon on the left  
  title: Text('My AppBar'),  // Title in the center  
  actions: [                  // Icons on the right
```

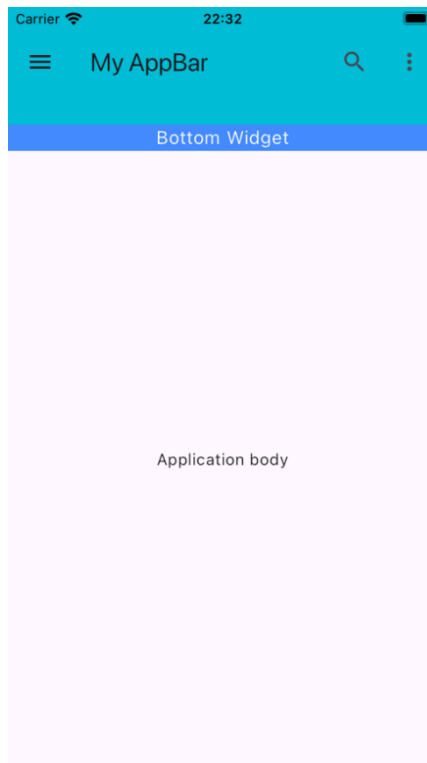
```

        IconButton(
          icon: Icon(Icons.search),
          onPressed: () {
            print('Search button pressed');
          },
        ),
        IconButton(
          icon: Icon(Icons.more_vert),
          onPressed: () {
            print('More button pressed');
          },
        ),
      ],
      bottom: PreferredSize(
        preferredSize: Size.fromHeight(50), // Height of the
        bottom
        child: Container(
          color: Colors.blueAccent,
          child: Center(
            child: Text(
              'Bottom Widget',
              style: TextStyle(color: Colors.white, fontSize:
16),
            ),
          ),
        ),
      ),
    ),
  ),
)

```

■ 動作説明:

- ◇ leading: 左側に メニューアイコン (Icons.menu) を表示.
- ◇ title: 中央に “My AppBar” を表示.
- ◇ actions: 右側に 検索アイコン (🔍) とオプションアイコン (⋮) を追加.
- ◇ bottom: AppBar の下部 に テキスト入りコンテナ を追加.



d. AppBar を使うべき場面

- AppBar がよく使われる場面:
 - ◇ アプリの統一感のあるトップバーが必要なとき.
 - ◇ タイトルやナビゲーションボタンを追加したいとき.
 - ◇ 検索,設定,その他のオプションを提供したいとき.
 - ◇ タブや追加コンテンツを AppBar の下に表示したいとき.

Flutter の Text ウィジェット

a. Text ウィジェットとは？

- Flutter アプリでテキストを表示するウィジェット.
- プレーンテキストとスタイル付きテキストの両方をサポート.
- 静的メッセージ,動的更新,レスポンスデザインに対応可能.

b. 主なプロパティ

- 主なプロパティ：

プロパティ	説明	使用例
data	表示するテキスト（必須）.	'Hello, Flutter!'
style	テキストの見た目を変更.	TextStyle(fontSize: 18, color: Colors.blue)

textAlign	テキストの配置(left, center, right).	TextAlign.center
maxLines	最大行数を制限.	maxLines: 2
overflow	テキストのオーバーフロー処理 (ellipsis, fade).	TextOverflow.ellipsis
softWrap	テキストの折り返し設定.	softWrap: true

c. 例：FlutterでのTextスタイルの設定

- 例のFlutterコード：

```
Text(
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,          // Sets the font size
    color: Colors.green,   // Text color
    fontWeight: FontWeight.bold, // Makes the text bold
  ),
  textAlign: TextAlign.center, // Centers the text
  maxLines: 2,                // Limits to 2 lines
  overflow: TextOverflow.ellipsis, // Adds "..." if the
text is too long
)
```

- 動作説明:
 - ◇ 'Welcome to Flutter!' → 表示するテキスト.
 - ◇ style → サイズ,色,太さなどを指定.
 - ◇ textAlign: TextAlign.center → テキストを中央揃え.
 - ◇ maxLines: 2 → 最大2行までに制限.
 - ◇ overflow: TextOverflow.ellipsis → 長すぎる場合“…”を表示.

d. なぜTextウィジェットを使うのか？

- Textウィジェットを使う理由:
 - ◇ 静的・動的テキストの表示が簡単.
 - ◇ 整列・折り返し・オーバーフロー処理が可能.
 - ◇ どんなレイアウトやUIデザインにも適用できる.

FlutterのContainerウィジェット

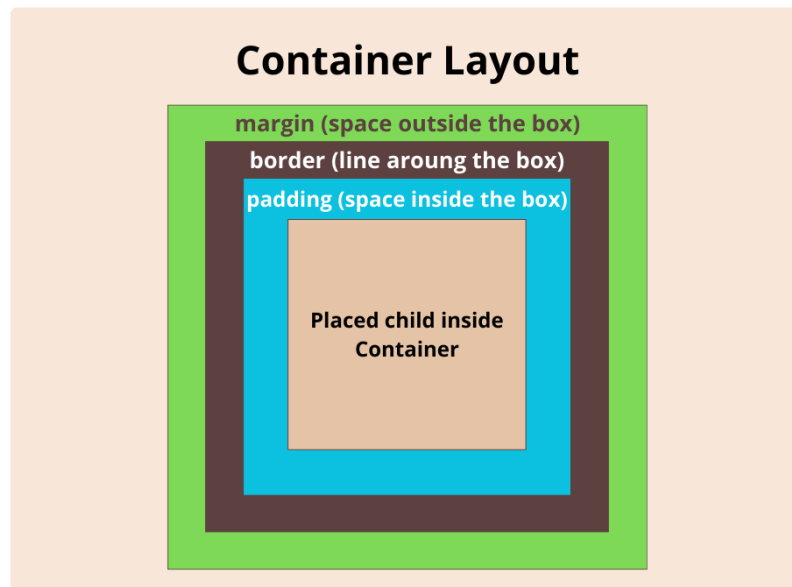
a. Containerウィジェットとは？

- スタイル,位置,レイアウトを設定できるカスタマイズ可能な「ボックス」.
- 他のウィジェットをラップし,間隔,色,装飾を適用するのによく使われる.
- Flutterで最も柔軟で一般的に使用されるウィジェットの1つ.

b. Container の主な特徴

■ 主なプロパティ：

プロパティ	説明	使用例
margin	外側のスペースを追加.	margin: EdgeInsets.all(10)
padding	内側のスペースを追加.	padding: EdgeInsets.all(15)
width & height	コンテナのサイズを設定.	width: 200, height: 100
color	背景色を設定.	color: Colors.blue
decoration	ボーダー, グラデーション, 影を追加.	decoration: BoxDecoration(...)



c. BoxDecoration を使った Container のカスタマイズ

■ BoxDecoration のプロパティ：

- ◇ color → 背景色を設定.
- ◇ border → ボーダーを追加.
- ◇ borderRadius → 角を丸くする.
- ◇ boxShadow → 影を追加.
- ◇ gradient → グラデーション背景を作成.

d. 例：Flutter でのスタイル付き Container

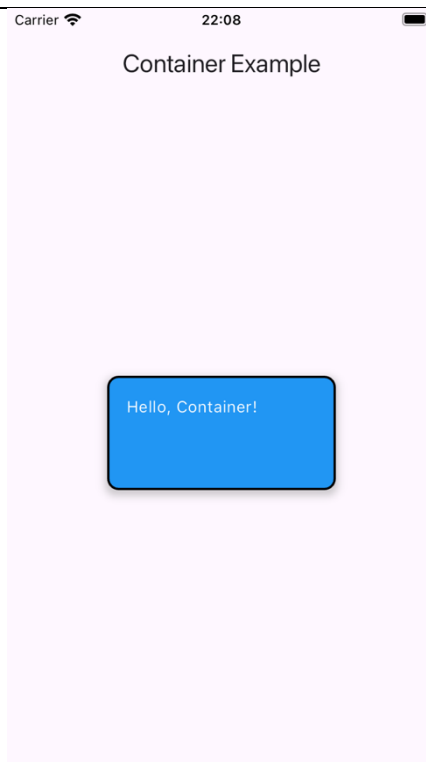
■ 例の Flutter コード：

```
Container(
  width: 200,           // Width of the container
  height: 100,          // Height of the container
```

```

margin: EdgeInsets.all(10), // Space outside the
container
padding: EdgeInsets.all(15), // Space inside the
container
decoration: BoxDecoration(
  color: Colors.blue,          // Background color
  border: Border.all(          // Adds a border
    color: Colors.black,
    width: 2,
  ),
  borderRadius: BorderRadius.circular(10), // Rounded
corners
  boxShadow: [                 // Adds shadow
    BoxShadow(
      color: Colors.grey.withOpacity(0.5),
      spreadRadius: 2,
      blurRadius: 5,
      offset: Offset(0, 3), // Shadow position
    ),
  ],
),
child: Text(
  'Hello, Container!', // Child widget inside the
container
  style: TextStyle(color: Colors.white),
),
)

```



- 動作説明:

- ◇ width & height → コンテナのサイズを設定.
- ◇ margin: EdgeInsets.all(10) → 外側のスペースを追加.
- ◇ padding: EdgeInsets.all(15) → 内側のスペースを追加.
- ◇ decoration → 背景色,ボーダー,影,角丸を適用.
- ◇ child → コンテナ内に Text ウィジェットを配置.

e. Container を使う理由

- Container ウィジェットのメリット:
 - ◇ margin と padding でスペースを追加.
 - ◇ ボーダー,色,影でスタイルを適用.
 - ◇ コンテンツのサイズや位置を調整できる.

Flutter の Row と Column ウィジェット

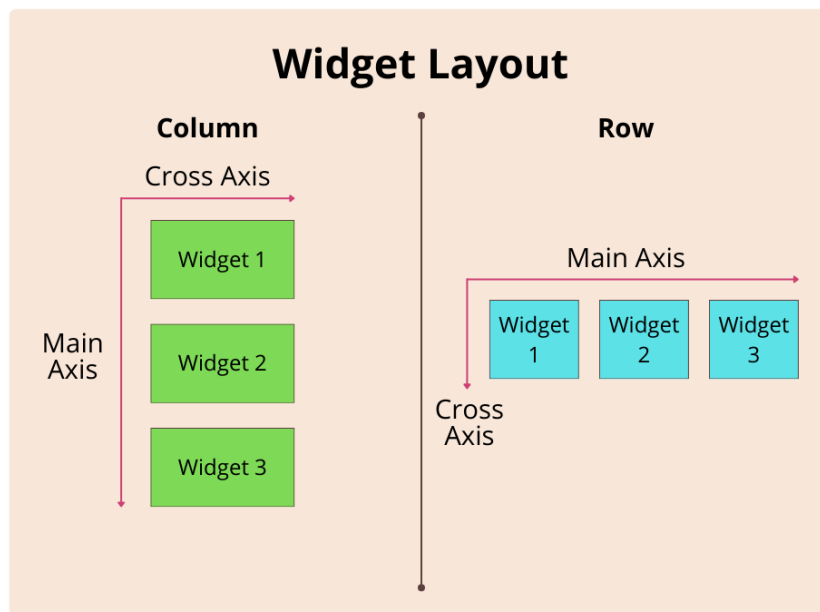
a. Row と Column とは？

- ウィジェットを水平または垂直に配置するためのウィジェット.
- Row → ウィジェットを横に並べる（水平）.
- Column → ウィジェットを縦に並べる（垂直）.

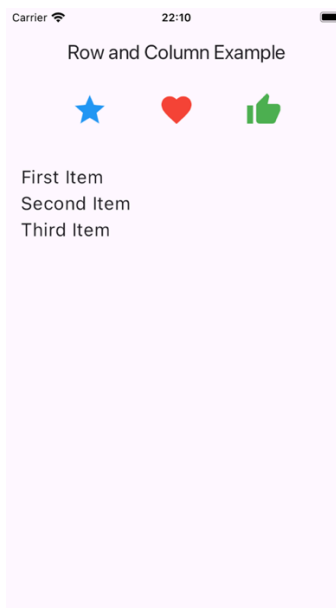
b. 主軸とクロス軸の理解

- Row や Column を使用する際の 2 つの重要な軸とは：

軸の種類	行	列
主軸 (Main Axis)	水平 (左⇄右) → ウィジェ ットを 横に並べる.	垂直 (上⇄下) → ウィジェ ットを 縦に並べる.
クロス軸 (Cross Axis)	垂直 (上⇄下) → 上・中 央・下に配置できる.	水平 (左⇄右) → 左・中 央・右に配置できる.



c. 例：Row と Column の実装例



■ 例の行コード：

```
// Row Example
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly, //
  Aligns widgets horizontally
  crossAxisAlignment: CrossAxisAlignment.center,    //
  Aligns widgets vertically
  children: [
    Icon(Icons.star, size: 40, color: Colors.blue),
    Icon(Icons.favorite, size: 40, color: Colors.red),
    Icon(Icons.thumb_up, size: 40, color: Colors.green),
  ],
)
```

```
),
```

- 動作説明

- ◇ `mainAxisAlignment.spaceEvenly` → アイコンを 水平方向 に均等配置.
- ◇ `crossAxisAlignment.center` → アイコンを 垂直方向 に中央配置.
- ◇ 3つのアイコンが横に並ぶ.

- 例の列コード：

```
// Column Example
Column(
  mainAxisAlignment: MainAxisAlignment.center,    //
  Aligns widgets vertically
  crossAxisAlignment: CrossAxisAlignment.start,  //
  Aligns widgets horizontally
  children: [
    Text('First Item', style: TextStyle(fontSize: 20)),
    Text('Second Item', style: TextStyle(fontSize: 20)),
    Text('Third Item', style: TextStyle(fontSize: 20)),
  ],
),
```

- 動作説明

- ◇ `mainAxisAlignment.center` → 垂直方向 に中央配置.
- ◇ `crossAxisAlignment.start` → 水平方向 に左揃え.
- ◇ 3つのテキストが縦に並ぶ.

d. Row と Column の違いはなぜあるのか？

- 違いまとめ：

特徴	行	列
配置方向	横 (水平) → 横に並べる.	縦 (垂直) → 縦に並べる.
主軸 (Main Axis)	水平 (左⇄右) → 間隔を調整.	垂直 (上⇄下) → 間隔を調整.
クロス軸 (Cross Axis)	垂直 (上⇄下) → 上・中央・下の位置調整.	水平 (左⇄右) → 左・中央・右の位置調整.

Flutter のボタンウィジェット

a. Flutter でのボタンは何か？

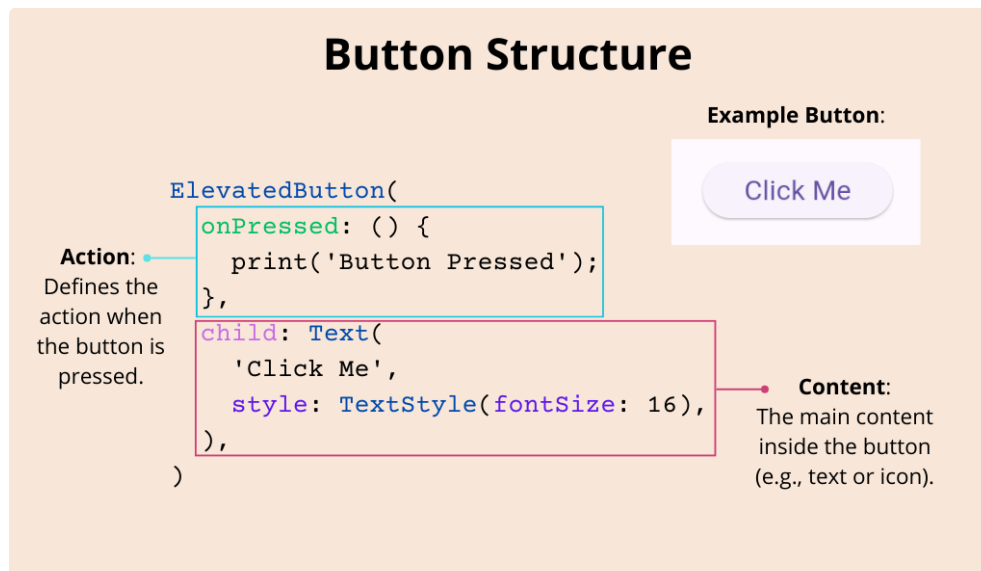
- クリック時にアクションを実行するインタラクティブなウィジェット.
- ナビゲーション, フォーム送信, ユーザー操作に不可欠.
- 用途に応じた異なる種類のボタンがある.

b. Flutter におけるボタンの種類

- Flutter は主に 3 種類のボタンを提供する:

ボタンの種類	概要	使用例
TextButton	仰角のないフラットなボタン.	目立たないアクション.
ElevatedButton	浮き上がったようなボタン.	重要なアクション.
OutlinedButton	枠線付き, 背景なしのボタン.	低優先度のアクション.

c. onPressed が重要な理由



- ボタンがクリックされたときの動作を定義する.
- onPressed がないとボタンは無効化される (グレーアウト) .
- 開発中でも onPressed を設定することが重要.

d. 例：Flutter でのボタンの使用例

```

Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    // TextButton Example
    TextButton(
      onPressed: () {
        print('TextButton clicked!');
      },
      child: Text('Text Button'),
    ),

    // ElevatedButton Example
    ElevatedButton(
      onPressed: () {
        print('ElevatedButton clicked!');
      },
      child: Text('Elevated Button'),
    ),
  ],
)

```

```
// OutlinedButton Example
OutlinedButton(
  onPressed: () {
    print('OutlinedButton clicked!');
  },
  child: Text('Outlined Button'),
),
],
)
```

e. ボタンの構造解説

- TextButton
 - 高さのないフラットなボタン.
 - 重要度の低いアクションに適している.
 - 例:「キャンセル」ボタンや補助的な操作.
- ElevatedButton
 - 表面から浮き上がって見えるボタン.
 - 主要なアクションを強調するために使用する.
 - 例:「送信」や「確認」ボタン.
- OutlinedButton
 - 枠線のあるボタン (背景なし) .
 - サブアクションや補助的な操作に適している.
 - 例:「詳細情報」や「もっと見る」ボタン.

f. Flutter でボタンを使う理由

- ユーザー操作 → アプリとのインタラクションを可能にする.
- カスタマイズ → 色,形,サイズなど自由に調整可能.
- 柔軟性 → 用途に応じた様々なボタンを利用できる.

まとめ

- Flutter でよく使われるウィジェットは,アプリの構造やスタイルを整えるのに役立つ.
- AppBar は,操作ボタンなどを備えた上部のナビゲーションバーを提供します.
- Text は,カスタマイズ可能なテキストを表示する.
- Container は,レイアウトやスタイリングに柔軟に対応できるボックス.
- Row と Column は,ウィジェットを水平方向・垂直方向に配置するために使用される.
- Button (ボタン) は,さまざまなスタイルでユーザーの操作を受け付けるために使われる.